Maîtrise du Hook useEffect et Synchronisation avec l'Extérieur

Objectifs

1. Isoler les effets secondaires de la logique pure de rendu en utilisant le Hook `useEffect`.
2. Configurer le tableau de dépendances pour exécuter un effet au montage, lors de la mise à jour de variables ciblées, ou une seule fois au démarrage.
3. Mettre en place des fonctions de nettoyage (*cleanup functions*) pour libérer des ressources persistantes (minuteurs, abonnements, écouteurs d'événements).
4. Éviter les boucles infinies de re-rendu et les fuites de mémoire.
5. Gérer les appels asynchrones à une API externe en respectant les bonnes pratiques d'encapsulation dans `useEffect`.

Contexte du Projet : OmniMonitor - Surveillance en Temps Réel

Vous enrichissez l'application **OmniMonitor** en connectant le tableau de bord à des flux de données en direct, en affichant une horloge système synchronisée et en mettant en place des alertes écoutant l'état des capteurs.

Étape 1 : Initialisation d'un Effet Unique au Montage (Chargement de Données API)

Vous allez simuler le chargement initial de la liste des stations météo depuis une API distante.

1. Créez un composant `src/components/StationManager.jsx`.
2. Importez `useState` et `useEffect` depuis `'react'`.
3. Déclarez un état local `stations` initialisé à un tableau vide `[]`, et un état de chargement `isLoading` initialisé à `true`.
4. Utilisez `useEffect` avec un **tableau de dépendances vide `[]`** pour simuler un appel réseau (`fetch` ou un `setTimeout` de 1,5 seconde) qui récupère les données et met à jour l'état.

5. Affichez un message `"Chargement des stations en cours..."` si `isLoading` vaut `true`, puis la liste des stations une fois chargées.

Étape 2 : Gestion des Dépendances et Synchronisation Ciblée

Vous allez configurer un effet qui réagit dynamiquement au changement d'une station sélectionnée.

1. Dans le même composant, ajoutez un état local `selectedStationId` (initialisé à `null`).
2. Ajoutez un second `useEffect` dont le tableau de dépendances contient `[selectedStationId]`.
3. À l'intérieur de cet effet, écrivez un code qui affiche dans la console : `"Chargement des détails pour la station ID : " + selectedStationId`.
4. Ajoutez des boutons pour changer l'identifiant de la station sélectionnée et observez dans la console le déclenchement ciblé de l'effet uniquement lorsque l'identifiant change.

Étape 3 : Implémentation d'une Fonction de Nettoyage (Cleanup)

Vous allez créer un composant d'horloge en temps réel qui met en place un minuteur et doit le nettoyer proprement lors de son démontage pour éviter les fuites mémoire.

1. Créez un composant `src/components/LiveClock.jsx`.
2. Déclarez un état local `time` initialisé à `new Date()`.
3. Dans un `useEffect` avec un tableau vide `[]`, configurez un minuteur (`setInterval`) qui met à jour l'heure toutes les secondes.
4. **Point critique :** Retournez une fonction de nettoyage (*cleanup function*) qui appelle `clearInterval(timerId)` pour stopper le minuteur lorsque le composant est démonté.
5. Dans le composant parent `App`, ajoutez un bouton pour afficher ou masquer le composant `LiveClock` (affichage conditionnel) et vérifiez dans la console que le message de nettoyage s'affiche bien au démontage.

Étape 4 : Gestion d'une Fonction Asynchrone dans `useEffect`

Vous allez corriger et structurer un appel asynchrone pour charger les détails d'une station.

1. Créez un effet qui doit exécuter une requête asynchrone au montage.
2. **Rappel de bonne pratique :** Ne déclarez jamais la fonction de rappel de `useEffect` directement comme `async`. Créez une fonction asynchrone interne (par exemple `async function fetchDetails()`) et appelez-la immédiatement à l'intérieur de l'effet.
3. Ajoutez un bloc `try / catch` pour gérer proprement les erreurs réseau et les consigner dans la console.

Critères de Validation

Avant de valider ce segment, vérifiez que :

- Le tableau de dépendances de chaque `useEffect` est correctement configuré selon le comportement attendu (vide pour le montage unique, ciblé pour les mises à jour).
- Aucun effet n'entraîne de boucle infinie de re-rendu (modification d'état sans dépendance ou avec dépendance cyclique).
- Les minuteurs et abonnements retournent systématiquement une fonction de nettoyage.
- Les fonctions asynchrones sont encapsulées proprement à l'intérieur de l'effet.